AI INFRA INTERVIEW HANDBOOK / 01

40 Q

强化学习八股

从策略梯度到 PPO / GRPO，再到 RL Infra

算法原理 · 奖励与评估 · Rollout 系统

采样与训练一致性 · 手写题 · 系统设计

> 公开面经筛题，原始论文与官方文档校验
>
> 每题包含：口述答案 / 背景 / 原理 / 实现 / 追问
>
> 附 CPU 参考代码、数值检查与可点击来源

检索与整理截至 2026-10-31

面向大模型算法、训练框架与 RL Infra 面试准备

独立技术整理 · 不代表任何公司的官方题库

阅读说明 / SCOPE & EVIDENCE

强化学习八股

这不是仅有题目和关键词的清单。每题按“能口述 → 能解释 → 能实现 → 能验证”的顺序组织，服务于大模型算法与 AI Infra 岗位中的连续追问。

检索范围与边界

本次跨平台检索并筛选了 17 条公开面经、转载与社区题库，主要可读材料来自牛客、GitHub、CSDN、博客园、AIInfraGuide 和独立技术站点。本册关联其中 11 条记录，并列出 35 项技术依据。不是对“全网所有面经”的穷尽抓取，也不宣称覆盖登录后、付费或未能公开访问的内容。

先按 RL / 后训练 / 训练框架 / 算子 / 分布式等关键词找候选，再区分作者自述、转载和题库；合并相同主题，最后用原始论文及官方文档写答案。未将转载次数转换为出题频率。

来源标签	如何解读
A · 作者自述	作者声称参加过对应面试。岗位、公司和面试日期仅按原帖记录，不是企业认证。
B · 转载 / 题库	用于发现问题和查漏补缺；不能当作独立面试样本，也不直接采信其中答案。
延伸题	为补全知识链或工程能力加入，未声称该具体题目在已检索面经中出现。

优先级与引用规则

本册 40 题中有 34 题标为 P0，其余为 P1。P0 表示建议先掌握的基础和岗位核心，P1 表示进阶补充；这是学习建议，不是统计学上的高频概率。题号中的“同类主题”只说明公开记录覆盖相近方向，不代表本册题目与原帖逐字相同。

每题底部链接到技术依据；来源目录保留原文链接、日期口径与证据等级。页面访问时间统一为 2026-10-31；只有月日的帖子不推定年份。官方 latest/main 页面可能继续变化，复现时请另锁定软件版本与 commit。

> **验证边界：**正文中的算量、显存与设备配置均写明假设。附录仅进行了 CPU 数学与代码测试，未运行多 GPU 训练、NCCL 性能或真实招聘流程；不提供虚构加速比。

建议读法

第一遍只看每题的“口述版”；第二遍推导公式并完成数值例子；第三遍遮住答案完成追问和代码。项目面试中，用真实日志、trace、对照实验与失败案例替代只背框架名。

目录 / CLICK TO NAVIGATE

问题导航

01 基础与数学

01 把大模型生成过程写成 MDP，状态、动作、奖励分别是什么？ 6
02 V、Q、Advantage 与 Bellman 方程怎样联系？ 7
03 策略梯度为什么能优化不可导的奖励？baseline 为什么不改期望？ 8
04 On-policy、Off-policy、Replay Buffer 与重要性采样有什么关系？ 9
05 GAE 如何推导？γ、λ、终止和截断怎么处理？ 10
06 DQN 为什么需要 replay 与 target network？Double DQN 改了什么？ 11
07 Actor-Critic、PPO、SAC 的关系，entropy 与 KL 怎么区分？ 12

02 后训练算法

08 预训练、SFT、RLHF、RLVR、DPO 分别解决什么？ 13
09 PPO 的 actor、critic、reward、reference 各干什么？如何走一轮？ 14
10 PPO clipping 的四种情况怎么解释？为什么不是双向硬裁剪？ 15
11 PPO 完整损失怎么写？critic target、value clipping、entropy 的关系？ 16
12 KL 方向与 k1/k2/k3 估计怎样区分？单样本 KL 为什么可能为负？ 17
13 GRPO 如何不用 critic 计算 advantage？与 PPO 的代价差别？ 18
14 GRPO 全对、全错、group 太小和奖励标准化怎么处理？ 19
15 Token mean、sequence mean、长度归一化为什么会改变训练目标？ 20
16 DPO 从 KL 正则化奖励优化如何推导？β 到底是什么？ 21
17 DPO 数据如何构造？为什么 loss 降了，实际效果却变差？ 22
18 ORM、PRM、规则奖励与 reward hacking 如何设计与排查？ 23
19 DAPO、Dr. GRPO、GSPO 应怎样比较，而不是只背缩写？ 24

目录 / CLICK TO NAVIGATE

问题导航 · 续

03 RL Infra 与一致性

20 π_current、π_old、π_rollout、π_ref 四个策略如何区分？ 25
21 训练与 rollout 的 logprob 对不上，从哪里查？ 26
22 MoE Router Replay 解决什么？为什么不等于训练完全确定性？ 27
23 异步 RL 为什么更快，又为什么更容易 off-policy？ 28
24 Importance correction、rejection sampling 与 PPO clipping 能否混用？ 29
25 训练权重如何同步到 vLLM/SGLang？重分片与 KV cache 怎么处理？ 30
26 verl、Ray、FSDP/Megatron、rollout engine 分别负责哪层？ 31
27 训练与 rollout 同卡部署还是分离部署？怎样做资源决策？ 32
28 Rollout 吞吐怎么优化？长尾、KV cache、group 与并发的权衡？ 33
29 Agentic RL 如何做多轮信用分配？工具结果为什么要 mask？ 34

04 诊断、评估与实验

30 奖励上涨但成功率下降，entropy collapse 或 KL 爆炸如何排查？ 35
31 RL 的显存账本与 OOM，为什么不能只算 actor 参数？ 36
32 分布式 RL 的 loss、advantage、gradient 应如何归约？ 37
33 RL checkpoint 应保存哪些状态？可重复性怎样分层验收？ 38
34 pass@1、pass@k、best-of-N 和训练 reward 应怎样比较？ 39
35 怎样设计 RL 消融实验，避免把系统加速说成算法提升？ 40
36 RL 的 batch、group、mini-batch、micro-batch 与 epoch 怎样调？ 41

05 手写、设计与项目

37 手写题：实现带 mask 的 PPO loss，要有哪些单元测试？ 42
38 手写题：GRPO advantage 与 DPO loss，怎么验证符号和分组？ 43
39 系统设计：8 张 GPU 上搭一套可验证的 GRPO 训练链路 44
40 项目深挖：如何把“我改过 RL 框架”讲成可验证的工程贡献？ 45

附录与来源

参考代码与边界说明 46
实际 CPU 验证结果 49
复习路径与模拟面试 50
公开面经与题库来源 51
论文与官方技术依据 53

检索结果如何变成题目

面经主题与本册映射

下表是主题索引，不是各家公司面试通过率或出题频率。回答内容经过重写和技术核验，没有复制某篇面经的完整答案。

知识模块	公开主题记录	题号
基础与数学	[I01]、[I02]、[I03]、[I04]	01–07
后训练算法	[I01]、[I02]、[I03]、[I04]、[I08]、[I09]、[I11]、[I12]、[I14]	08–19
RL Infra 与一致性	[I01]、[I02]、[I08]、[I12]、[I13]、[I14]	20–29
诊断、评估与实验	[I01]、[I02]、[I08]、[I11]、[I13]、[I14]	30–36
手写、设计与项目	[I01]、[I02]、[I04]、[I07]、[I09]、[I12]、[I13]	37–40

PPO → GAE → GRPO

解释估计器，再说明训练流水线与边界条件。 同类主题：[I01] [I02]。

DPO → β → 数据质量

固定 reward 解释与实际有限数据优化不能混用。 同类主题：[I04] [I09]。

RL 系统 → 异步 → 一致性

先追溯样本来自哪个策略，再讨论 correction。 同类主题：[I13] [I14]。

Agentic RL → 稀疏奖励 → 工具

同时讨论信用分配、mask 与环境可靠性。 同类主题：[I01] [I12]。

> 同一作者的不同面试保留为不同经历，但不当成独立作者样本；多平台搬运只作为一个线索。帖子中的薪资、岗位排名、通过率以及未经证实的“必考”结论未纳入本册。

01 基础与数学 / QUESTION 01

01 把大模型生成过程写成 MDP，状态、动作、奖励分别是什么？

P0 · 优先掌握 公开记录同类主题：[I01] [I03]

> **面试口述版**
>
> 单轮语言模型可把 prompt 与已生成前缀视为状态，下一个 token 视为动作，模型条件分布就是策略；奖励可以只在回答结束时出现。Agent 多轮任务还要纳入工具观察和环境变化，不能只把“整段回答”粗略当成一个固定分类标签。

背景：问题为什么会出现

面试官先确认你能把 RL 术语对应到真实张量。记号：x 是问题，y 是回答，s_t=(x,y_<t)，a_t=y_t，πθ(a_t|s_t) 是 softmax 概率；回报 G_t 汇总从 t 开始的奖励。对有限长度回答可选 γ=1，但这属于任务建模选择，而不是 RL 的统一规定。

原理与推导：把关键条件说清楚

纯文本自回归里，给定 token 后的状态转移主要是追加前缀；外部工具调用则会产生新的 observation。上下文窗口截断可能丢失真实状态，因此对真实任务更准确的表述可能是部分可观测过程。把工具输出当观察，不能把它的所有 token 都视为 actor 自己选择的动作。

$$J(\theta) = E_{\tau\sim\pi\theta}[\Sigma_t \gamma^t r_t] ; \pi\theta(y|x) = \Pi_t \pi\theta(y_t|x,y_{<t})$$

实现方案与验证

落地时为每条轨迹保存 prompt/response token、生成 token mask、终止原因、奖励、behavior logprob、模型与环境版本。先用一个两步任务检查“动作→奖励→回报”的索引，再扩展到多轮。连续工具观察应正确进入后续条件概率，却不直接进入 actor 的动作 loss。

常见追问与易错点

追问：为什么还需要 reference？答：它不是 MDP 必需组件，而是对策略漂移施加约束的工程/优化设计。误区：把监督答案当 reward；答案可以供 verifier 判分，但 reward 本身是评价结果。



工具返回是 observation；只有 actor 生成的动作进入相应 policy loss。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R01] [R16] [R24] | 口述、数值例子及实验建议为本册组织与推导。

01 基础与数学 / QUESTION 02

02 V、Q、Advantage 与 Bellman 方程怎样联系？

P0 · 优先掌握 公开记录同类主题：[I01] [I02]

> **面试口述版**
>
> V 是在状态下继续执行策略的期望回报；Q 额外指定第一步动作；A=Q−V 衡量这个动作相对该状态平均水平好多少。优势不是“是否答对”的同义词，而是经过基线校正的相对收益。

背景：问题为什么会出现

同样获得 1 分，在一个几乎必答对的题和一个基本不会的题上，学习价值不同。价值函数建立状态条件基线；critic 的输入通常是前缀隐状态，输出每个动作时刻之前的价值估计。

原理与推导：把关键条件说清楚

Bellman 展开把长回报拆成当前奖励加下一状态价值。TD residual δ_t=r_t+γV(s_{t+1})−V(s_t) 是一步估计；若 V 正确，条件于当前状态和动作的 δ 期望等于 advantage。举例 γ=0.9、r=1、V(next)=2、V(now)=2.5，则 δ=0.3，说明比现有基线更好。

$$V^{\pi}(s)=E_a[Q^{\pi}(s,a)]\ ;\ Q^{\pi}(s,a)=E[r+\gamma V^{\pi}(s')]\ ;\ A^{\pi}=Q^{\pi}-V^{\pi}$$

实现方案与验证

实现时明确 V_t 对应“采取 token t 之前”还是之后，保证 shift 对齐。critic target 可以是 λ-return=Â_t+V_old,t；把 target detach，避免目标和预测互相追逐。记录 value loss、解释方差、不同长度的 value 偏差，不能只看 policy reward。

常见追问与易错点

追问：奖励全非负，advantage 能为负吗？答：能，低于基线就为负。追问：critic loss 下降代表策略进步吗？答：不代表，预测常数也可能把低方差标签拟合好，仍需独立策略评估。

技术依据：[R01] [R05] ｜ 口述、数值例子及实验建议为本册组织与推导。

01 基础与数学 / QUESTION 03

03　策略梯度为什么能优化不可导的奖励？baseline 为什么不改期望？

P0 · 优先掌握　公开记录同类主题：[I01] [I02]

> **面试口述版**
>
> 不需要对奖励函数求导，而是对采样轨迹的概率求导：高回报动作增加 logprob，低回报动作减少 logprob。对状态依赖、动作无关的 baseline，期望梯度不变，但方差可下降。

背景：问题为什么会出现

代码判题、人工偏好、网页工具都可能不可微。RL 的入口是模型对动作概率的可微性，而不是要求环境端到端可微。REINFORCE 的主要困难是噪声与长程信用分配。

原理与推导：把关键条件说清楚

由 ∇p=p∇log p 可把 ∇E[R] 写成 E[R∇logπ]。状态固定时，Σ_a π(a|s)∇logπ(a|s)=∇Σ_aπ(a|s)=0，因此减 b(s) 不改变该条件期望。成立条件包括 baseline 不随当前采样动作变化，且不要让 actor loss 通过 baseline 反向传播。

$$\nabla\theta\, J = E[\Sigma_t \nabla\theta \log \pi\theta(a_t|s_t)\cdot(G_t - b(s_t))]$$

实现方案与验证

损失可以写为 −stopgrad(A)・logπ(a|s)。先在二动作 bandit 上验证正 advantage 提高对应动作概率；再比较无 baseline 与状态 baseline 的梯度方差。不要用 loss 的绝对正负判断训练好坏，策略梯度 loss 可以为负。

常见追问与易错点

追问：同一 GRPO group 的均值也包含自己，仍是严格动作无关 baseline 吗？答：不是简单套用该证明；有限组样本会带来自依赖与缩放等效应。leave-one-out 可去掉自身项，但也改变估计器，需要单独分析。

技术依据：[R02] [R03]　|　口述、数值例子及实验建议为本册组织与推导。

01 基础与数学 / QUESTION 04

04 On-policy、Off-policy、Replay Buffer 与重要性采样有什么关系？

P0 · 优先掌握 公开记录同类主题：[I02] [I03] [I04]

> **面试口述版**
>
> on-policy 依赖当前或近似当前策略生成的数据；off-policy 可学习其他行为策略的数据。重要性采样用目标概率除行为概率修正分布，但长序列连乘会有高方差；截断比值是偏差与方差的取舍，不是免费修正。

背景：问题为什么会出现

LLM 中“离线数据”和“off-policy 算法”不是完全同义。PPO 通常对一批新 rollout 做有限轮更新，随后重新采样；数据被多次使用后已不严格来自更新中的当前策略，但不因此等价于可以任意复用历史数据的 off-policy 算法。

原理与推导：把关键条件说清楚

若 a~μ，Eπ[f(a)]=Eμ[(π(a)/μ(a))f(a)]，要求 π 的支持集被 μ 覆盖。轨迹级修正是各步比值乘积；逐 token 修正只直接校正条件动作分布，不能无条件解决历史状态分布偏差。temperature、top-p 与 top-k 也属于实际行为分布定义。

```
w(a)=π(a|s)/μ(a|s) ; ESS=(Σ_i w_i)^2 / Σ_i w_i^2
```

实现方案与验证

缓存数据时保留采样时 logprob、采样配置与权重版本。对 logratio、ESS、clip fraction、样本年龄建监控；为数据设最大滞后版本和过期策略。若日志只存未加采样变换的 logits 概率，应明确修正是近似，而非宣称严格无偏。

常见追问与易错点

追问：top-k 把某动作概率置零，还能靠 IS 恢复吗？答：不能从从未采到的动作恢复其贡献。追问：PPO 的 old policy 等于 reference 吗？答：通常不等；前者是本轮近端优化基准，后者是跨轮约束锚点。

技术依据：[R01] [R03] [R20] | 口述、数值例子及实验建议为本册组织与推导。

01 基础与数学 / QUESTION 05

05 GAE 如何推导？γ、λ、终止和截断怎么处理？

P0 · 优先掌握 公开记录同类主题：[I01] [I02]

> **面试口述版**
>
> GAE 是 TD residual 的指数加权和：λ 小更依赖 critic、方差低但可能偏；λ 大更接近长回报、方差高。实现中必须区分真实终止与外部截断：前者通常不 bootstrap，后者通常应利用截断前的最终 observation。

背景：问题为什么会出现

大模型任务常把结果奖励放在最后一个生成 token。直接给每步整段 reward 信号噪声大，GAE 结合价值预测分配各时间步优势。γ 控制未来回报权重，λ 控制使用多远的 TD residual，不是同一个参数。

原理与推导：把关键条件说清楚

递推为 A_t=δ_t+γλc_t A_{t+1}；δ_t=r_t+γb_t V(next)−V_t。b_t 控制是否 bootstrap，c_t 控制是否跨当前轨迹段继续递推。真实结束时两者为 0；采样器时间截断时可取 b=1、c=0，避免把 reset 后新 episode 的残差串进来。若任务本身把长度上限定义为失败终点，则终止语义不同。

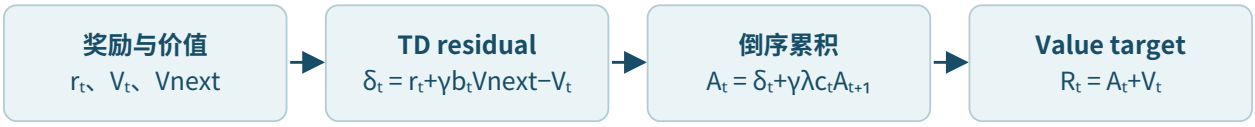
```
δ_t=r_t+γ b_t V_{t+1}−V_t ; Â_t=δ_t+γλ c_t Â_{t+1}
```

实现方案与验证

用倒序循环计算，padding 不参与；先 detach values 与 advantages，再按约定归一化。测试三组：无中间奖励的终止轨迹、外部 truncation、两段拼接轨迹。附录代码显式传 bootstrap 与 continuation 两个 mask，不把一个 done 变量包办所有语义。

常见追问与易错点

追问：λ=1 一定无偏吗？答：完整终止轨迹和正确边界下可接近 Monte Carlo；存在截断、bootstrap 或不正确价值时不能笼统承诺。追问：γ=1 可以吗？答：有限时域可以；长短答案的隐式权重仍受 loss reduction 影响。



bootstrap mask b 与跨段递推 mask c 分开：termination ≠ truncation。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R05] [R06] ｜ 口述、数值例子及实验建议为本册组织与推导。

01 基础与数学 / QUESTION 06

06 DQN 为什么需要 replay 与 target network？Double DQN 改了什么？

P1 · 进阶补充 延伸题 · 知识链与工程补全

> **面试口述版**
>
> DQN 用 Q 网络预测离散动作价值；经验回放降低时序相关并复用数据，目标网络让 bootstrap 目标变化更慢。Double DQN 分离选动作和估值，缓解 max 对噪声的过估计。它们并不等于 PPO 的 old/reference 机制。

背景：问题为什么会出现

这是经典 RL 的基础追问，即使岗位偏 LLM，也应能解释 value-based 与 policy-based 的区别。DQN 的目标把即时奖励与下一状态最大 Q 相加；同时拟合自己生成的目标容易不稳定。

原理与推导：把关键条件说清楚

普通目标 y=r+γ max_a Q_target(s′,a)。Double DQN 改成先用 online 网络求 argmax，再用 target 网络对该动作估值。比如 online 估计 [5,4]、target 为 [2,3]：普通目标取 3，Double 用 online 选中的动作取 2。它减少某类偏差，但不保证所有环境收敛。

```
y_Double=r+γ Q_target(s′, argmax_a Q_online(s′,a))
```

实现方案与验证

实现应屏蔽 terminal bootstrap、detach target、检查 replay 采样分布与 target 更新周期。若使用 prioritized replay，还需说明重要性权重的目的与近似。把两动作小环境跑通，比只背“off-policy”有说服力。

常见追问与易错点

追问：为何 LLM 不直接用 DQN？答：词表动作大、长前缀状态空间巨大、序列奖励稀疏且已有强策略模型，直接套 max-Q 和 replay 并不自然；这是工程与学习稳定性的取舍，不是数学上禁止。

技术依据：[R07] [R08] ｜ 口述、数值例子及实验建议为本册组织与推导。

01 基础与数学 / QUESTION 07

07 Actor-Critic、PPO、SAC 的关系，entropy 与 KL 怎么区分？

P1 · 进阶补充 公开记录同类主题：[I01] [I03]

> **面试口述版**
>
> Actor-Critic 是结构范式，PPO 是常见的近端策略更新方法，SAC 是最大熵、通常 off-policy 的 actor-critic 算法。entropy 鼓励策略自身多样性，KL 则比较两个分布；两者作用不能混成“都是防过拟合”。

背景：问题为什么会出现

LLM 岗常从 PPO 追问到更基础的探索问题。熵 $H(\pi)=-\Sigma\pi\log\pi$ 只取决于当前分布；$KL(\pi||\pi_{ref})$ 还取决于参考分布。如果 reference 均匀，二者才在常数意义上关联。

原理与推导：把关键条件说清楚

SAC 通过奖励加温度加权熵优化软价值与策略，常配合双 Q 网络和 replay。连续控制通常用可重参数化策略；离散版本也存在，因此不能说 SAC “只能连续动作”。PPO 的 clipping 约束与 entropy bonus 是两件事，不能把增大 clip 范围当成直接提高探索。

实现方案与验证

LLM 实验中分开记录 conditional token entropy、到 reference 的 KL、结果多样性与正确率。一个模型可以 entropy 很高却充满无意义输出，也可以 KL 很小但持续重复。优先在同一任务与采样预算下比较，而不是用单个统计量作算法排名。

常见追问与易错点

追问：降低 temperature 就等价于训练更低熵吗？答：不是，采样分布变了但模型参数未必变化；还会影响 behavior logprob 和训练分布。追问：PPO 和 SAC 谁一定更好？答：由动作空间、交互成本、数据复用与稳定性要求决定。

技术依据：[R09] [R04] [R01] ｜ 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 08

08 预训练、SFT、RLHF、RLVR、DPO 分别解决什么?

P0 · 优先掌握 公开记录同类主题：[I01] [I04] [I12]

> **面试口述版**
>
> 预训练学习文本分布与广泛能力，SFT 学习示范行为；RLHF 用偏好或奖励信号优化策略，RLVR 依赖可验证结果反馈；DPO 是直接使用偏好对的优化目标，不必在线训练一个显式 reward model 和 critic。它们不是互斥的模型结构。

背景：问题为什么会出现

“为什么 SFT 后还要 RL”应从优化目标回答：SFT 提高示范回答的似然，却不直接比较模型自行采样轨迹的任务收益；RL 能把生成过程和奖励联系起来，但奖励设计、探索和分布漂移会引入新问题。

原理与推导：把关键条件说清楚

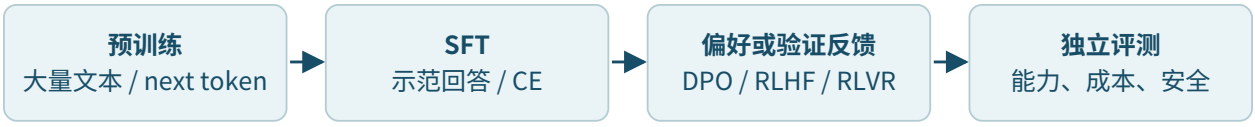
正确数据形态不同：SFT 通常用 prompt→目标回答；DPO 用同 prompt 的 chosen/rejected；GRPO 可用 prompt 加 verifier/参考答案，不要求离线给出每条完整 reasoning。GRPO 的候选轨迹由策略采样产生；“训练集必须有人工思维链”不是普遍前提。

实现方案与验证

方案选择先看反馈：有高质量示范先做 SFT；有偏好对可比较 DPO；有低成本正确性判据可探索 RLVR；不可验证的开放问答更依赖奖励模型与人评。比较时固定基模、数据范围、生成预算和 held-out 评估，避免把基模差异当算法收益。

常见追问与易错点

追问：RL 一定创造全新能力吗？答：仅凭某个 benchmark 提升无法证明；需区分采样概率重分配、探索、计算预算和污染。追问：RLVR 不需要人工吗？答：verifier、数据清洗和评测仍需设计与审查。



路线不是强制串行；训练配方应由数据形态和反馈可靠性决定。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R10] [R12] [R16] | 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 09

09 PPO 的 actor、critic、reward、reference 各干什么？如何走一轮？

P0 · 优先掌握 公开记录同类主题：[I01] [I03] [I08]

> **面试口述版**
>
> Actor 生成并更新策略；critic 估计状态价值；reward model 或规则给反馈；reference 提供漂移约束。一次迭代通常是采样、评分与 logprob/value 计算、构造 advantage/return、若干 mini-batch 更新，再同步新权重。四个角色不必都是独立同尺寸大模型。

背景：问题为什么会出现

“PPO 需要四个模型”的说法指常见 RLHF 组织方式，不是 PPO 数学定义。规则奖励可替代神经 reward；value head 可以共享部分结构；reference 常冻结，但这些选择各有显存与耦合代价。

原理与推导：把关键条件说清楚

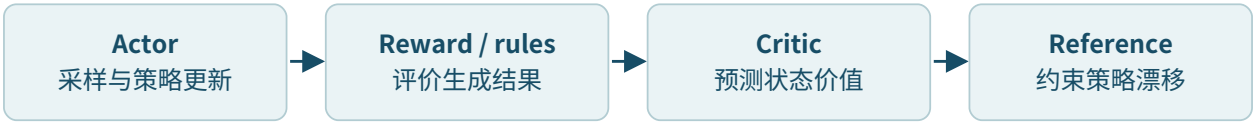
采样时记录 old/behavior logprob；训练当前 actor 得到 new logprob；KL 可能作为 token reward 的惩罚，也可能单独放入 loss。critic 使用固定 target 学习，actor 使用固定 advantage。不能不加说明地同时在 reward 与 loss 重复施加同一 KL 惩罚。

实现方案与验证

将流水线切成 rollout、reward、old/ref/value、advantage、actor/critic update、weight sync 六段计时。按角色计算参数、梯度、优化器、激活和 KV 峰值；冻结角色只节省梯度/优化器，不代表它的权重与前向完全免费。

常见追问与易错点

追问：GRPO 省掉了什么？答：经典形式省掉显式 critic/value 训练，增加同 prompt 多候选采样成本；reference/KL 是否保留由配方决定。追问：old model 必须额外占一份 GPU 权重吗？答：可保存 logprob 或重算，取决于实现和更新次序。



四种职责，不等于必须有四个同等尺寸的独立大模型。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R10] [R22] [R03] | 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 10

10 PPO clipping 的四种情况怎么解释？为什么不是双向硬裁剪？

P0 · 优先掌握 公开记录同类主题：[I02] [I03]

> **面试口述版**
>
> PPO 最大化 min(rA,clip(r)A)。A 为正时阻止概率比过度增大带来的额外收益；A 为负时阻止概率比过度减小带来的额外收益。相反方向仍保留惩罚，因此不能把 r 超范围的所有梯度都清零。

背景：问题为什么会出现

概率比 r=π_new/π_old，ε 控制近端区间。它是采样动作的 surrogate 目标，并不是逐参数信赖域硬约束，也不是直接把新策略概率投影到某个区间。

原理与推导：把关键条件说清楚

例：ε=0.2。A=+2、r=1.4，目标取 min(2.8,2.4)=2.4，向好方向被封顶；A=+2、r=0.6，取 min(1.2,1.6)=1.2，仍惩罚概率下降。A=−2、r=0.6，取 min(−1.2,−1.6)=−1.6；A=−2、r=1.4，取 min(−2.8,−2.4)=−2.8，仍惩罚坏动作概率增加。

$$L^{clip} = E[\min(r_t \hat{A}_t, clip(r_t,1-\varepsilon,1+\varepsilon)\hat{A}_t)]$$

实现方案与验证

最小化时取负号，或用 max(−rA,−clip(r)A)，不要把 min/max 反写。计算 logratio 用 FP32，old_logprob 与 advantage detach。对四种输入做数值/梯度测试，再记录 clip fraction 与 approx KL；clip fraction 高可能是学习率、epoch 数或训练采样不一致导致。

常见追问与易错点

追问：clipped 样本概率之后就不再变吗？答：不保证，共享参数会被其他 token、KL 和辅助损失更新。追问：ε 越小越安全？答：更保守可能学习停滞，且 clipping 不是全分布 KL 的硬上界。

优势符号	概率比偏低	概率比偏高
A > 0	r < 1−ε：保留梯度	r > 1+ε：目标封顶
A < 0	r < 1−ε：目标封顶	r > 1+ε：保留梯度

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R03] [R04] ｜ 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 11

11 PPO 完整损失怎么写？critic target、value clipping、entropy 的关系？

P0 · 优先掌握 公开记录同类主题：[I01] [I08]

> **面试口述版**
>
> actor surrogate、value regression、entropy bonus 与可能的 KL 项要分别定义，再按系数组合。value target 通常来自 return/GAE，必须固定；value clipping 是某些实现的稳定化选项，不是所有 PPO 都必需。

背景：问题为什么会出现

论文式目标常用最大化写法，工程代码常用最小化 loss，符号容易混。actor 的 advantage 估计、critic 的回归目标、reference KL 的来源要分别解释，不要只抄一条混合公式。

原理与推导：把关键条件说清楚

一种最小化约定为 L=−Lclip+c_v · MSE(V,R_target)−c_h · H+β · KL。若 KL 已用于构造 r_t，再额外加 KL 应说明新的目标含义。R_target 可用 A_GAE+V_old；并不是简单把整条轨迹的末端 reward 无差别当作每个时刻价值标签。

$$R_{target,t} = \mathrm{stopgrad}(\hat{A}_t + V_{old,t}) \; ; \; L = -L_{clip} + c_v L_v - c_h H + \beta KL$$

实现方案与验证

实现时分开 optimizer 或明确共享参数；保留 mask；监控 policy loss/value loss 的量纲与梯度范数。训练不稳先把单条轨迹算对，再检查 old 值是否在一个 PPO epoch 内错误刷新、是否误把 padding 或 prompt 带进 actor loss。

常见追问与易错点

追问：actor 与 critic 学习率为什么不同？答：二者目标噪声、监督密度和梯度尺度不同。追问：value loss 很大就提高其权重吗？答：先排查 target 尺度、奖励缩放与边界；提高权重可能让共享骨干被 critic 主导。

技术依据：[R03] [R05] [R04] ｜ 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 12

12　KL 方向与 k1/k2/k3 估计怎样区分？单样本 KL 为什么可能为负？

P0 · 优先掌握　公开记录同类主题：[I02] [I08] [I14]

> **面试口述版**
>
> 先说采样来自哪个分布。若 a~p，KL(p||q)=E_p[log p−log q]；单样本 log 比可以为负，期望才非负。k2 是局部近似，不是任意两分布的精确 KL；k3 的无偏性也依赖采样与支持集条件。

背景：问题为什么会出现

用 u=q(a)/p(a)：k1=−log u，k2=(log u)²/2，k3=u−1−log u。因为 log u≤u−1，k3 每点非负；若 p、q 在同一支持集上归一化，E_p[u−1]=0，因此 k3 与 k1 期望相同。

原理与推导：把关键条件说清楚

原创二动作检查：p=(0.8,0.2)，q=(0.5,0.5)，精确 KL=0.8ln1.6+0.2ln0.4≈0.1927。第二个动作的 k1≈−0.9163，却不违反 KL 非负。若实际从旧策略 μ 采样，直接平均 current-vs-ref 的 k3 不自动等于 KL(current||ref)。

```
a~p ; u=q(a)/p(a) ; k1=−ln u ; k2=(ln u)^2/2 ; k3=u−1−ln u
```

实现方案与验证

在日志里明确写 estimator、采样分布、按 token 还是 sequence 聚合、mask 和是否重要性加权。使用 double/FP32 参考检查接近一致与极端概率情形。top-k 截断会改变支持集；数值 clamp 防溢出也会改变估计偏差。

常见追问与易错点

追问：k3 无偏就代表用它反传也是目标 KL 的无偏梯度吗？答：不自动成立；样本分布的参数依赖和 stop-gradient 处理会改变梯度估计。面试时先区分“数值估计器”与“用什么 surrogate 更新策略”。

技术依据：[R11] [R12] [R20]　|　口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 13

13 GRPO 如何不用 critic 计算 advantage？与 PPO 的代价差别？

P0 · 优先掌握 公开记录同类主题：[I01] [I02] [I09]

> **面试口述版**
>
> GRPO 对同一 prompt 采样 G 个回答，用组内奖励相对值构造 advantage，再优化带 ratio/clipping 的策略目标。经典 outcome-reward 形式把同一回答的优势分给其生成 token，省掉 critic，但付出多候选生成和组内统计成本。

背景：问题为什么会出现

对奖励 R_1…R_G，典型 Â_i=(R_i−mean(R))/(std(R)+ε)。它衡量的是该回答相对同题其他回答的好坏，不是跨所有题统一校准的 value。不同实现对 std、token/sequence reduction 和 KL 的定义可能不同。

原理与推导：把关键条件说清楚

示例 G=4，奖励 [0,0,1,1]，采用总体标准差得到 mean=0.5、std=0.5，优势 [-1,-1,1,1]。同样的正确奖励在不同组内可产生不同梯度尺度；只有 prompt 分组正确，组相对比较才有意义。group 不应在打乱/切微批后被意外混合。

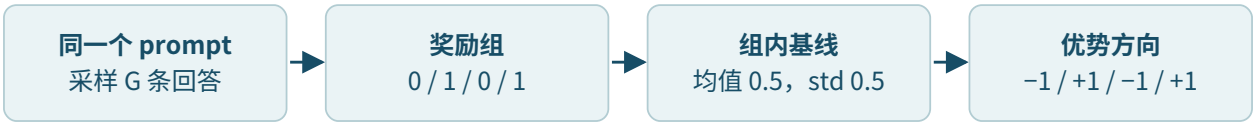
$$\hat{A}_i=(R_i-\mu_{group})/(\sigma_{group}+\varepsilon)\ ;\ i=1,\dots,G$$

实现方案与验证

保留 prompt/group ID；在 advantage 生成后再允许 minibatch shuffle。检查 G=1、零方差、被截断回答、分布式跨 rank 分组。先与无标准差归一化或 leave-one-out 基线做对照，而不是以算法名称推断完全相同实现。

常见追问与易错点

追问：GRPO 总比 PPO 省显存吗？答：去掉 critic 状态有优势，但 G 条 rollout 的 KV 和缓存可显著增加峰值；应按阶段测。追问：必须有正负答案标签吗？答：不必是离线偏好对，只要有采样回答的可用奖励。



示意忽略 ε；整组同奖励时优势项为零，独立 KL 项未必为零。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R11] [R13] [R26] ｜ 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 14

14 GRPO 全对、全错、group 太小和奖励标准化怎么处理?

P0 · 优先掌握 公开记录同类主题：[I02] [I09]

> **面试口述版**
>
> 全对或全错且奖励完全相同，会使组相对 advantage 为零，结果奖励对应的 policy gradient 没信号；独立 KL 或其他项仍可能更新。不能把“这个 group 没有效奖励梯度”误说成“整个模型无梯度”。

背景：问题为什么会出现

小 G 产生噪声大的均值与方差，大 G 增加 rollout 成本且不保证样本独立。按 std 除法会改变不同难度 prompt 的相对权重；奖励方差很小而不为零时，也可能放大噪声。

原理与推导：把关键条件说清楚

一个简单对照：两个问题分别奖励 [0,0,0,1] 和 [0,0,1,1]，被标准化后的单个正样本权重不同。动态筛选非零方差组能提高每次更新的信息密度，但丢弃简单/极难题会改变训练分布，还要计算被丢弃 rollout 的成本。

实现方案与验证

记录 zero-variance group 比例、每组 reward spread、有效 prompt 数与生成 token 总量。可调整课程难度、采样温度、group 大小，或比较不除 std 的估计器；加入过程奖励必须验证其可信度。不要通过给随机噪声奖励“制造方差”。

常见追问与易错点

追问：全部答错是否说明 RL 学不了？答：当前奖励无法区分这些回答；可以改任务难度、冷启动或判分粒度，而不是证明永久不能学习。追问：把 zero-advantage token 删掉能节省什么？答：训练计算可能减少，生成成本已经支付。

技术依据：[R11] [R17] [R18] | 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 15

15 Token mean、sequence mean、长度归一化为什么会改变训练目标?

P0 · 优先掌握 公开记录同类主题：[I02] [I11]

> **面试口述版**
>
> “先每句平均再全组平均”和“所有有效 token 一起平均”通常不等价。前者每条回答权重相近，后者长回答贡献更多 token；对 outcome advantage，把长度除在哪里会改变梯度和长度偏好。

背景：问题为什么会出现

令每 token 损失为 ℓ_it。sequence mean 是 (1/N)Σ_i[(1/L_i)Σ_tℓ_it]；token mean 是 Σ_itℓ_it/Σ_iL_i。两条长度 2 与 8 的回答：前者每条总权重 1/2，后者分别 1/5 与 4/5。这是目标选择，不只是张量写法。

原理与推导：把关键条件说清楚

还要区分训练 loss 的 reduction、报告 metric 的 reduction、跨 DP rank 的 reduction。不同 rank 长度不同，平均各 rank 的 token 均值一般不等于全局 token 均值。prompt、padding、工具返回文本需根据任务 mask；生成的 EOS 是否计入应统一。

$$L_{seq} = (1/N)\Sigma_i (1/L_i)\Sigma_t \ell_{it} ; L_{token} = \Sigma_{i,t} \ell_{it} / \Sigma_i L_i$$

实现方案与验证

先写出期望优化的目标，再用两条不同长度样本手算。分布式实现累计 numerator 和有效 token denominator；如果 DDP 默认平均梯度，可按全局有效 token 数调整本地 loss 系数。对 token masking 的测试应检查被屏蔽位置梯度为零。

常见追问与易错点

追问：较长回答一定更好或更差吗？答：没有普遍结论；应报告正确率与长度/预算关系。追问：Dr.GRPO 是否等于只改一个参数？答：依赖具体代码配方，需逐项核对 advantage 标准化与 loss 分母，不能只看方法名。

技术依据：[R13] [R17] [R18] ｜ 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 16

16 DPO 从 KL 正则化奖励优化如何推导？β 到底是什么？

P0 · 优先掌握 公开记录同类主题：[I04] [I09] [I12]

> **面试口述版**
>
> 对固定奖励的 KL 正则化目标，最优策略满足 π*∝π_ref · exp(r/β)。把奖励用策略对数比重写并代入偏好模型，就得到 DPO 的二元偏好 loss。β 同时出现在该正则化推导与优化梯度尺度中，不能简单说“越大越偏离 reference”。

背景：问题为什么会出现

设目标为 max_π Eπ[r]−β KL(π||π_ref)。用归一化约束做变分，得 r=βlog(π*/π_ref)+βlogZ(x)。同一 prompt 的 chosen 与 rejected 做差时 Z 消掉，再使用 Bradley–Terry 偏好概率 sigmoid(r_w−r_l)。

原理与推导：把关键条件说清楚

DPO 的 Δ=logπθ(y_w|x)−logπθ(y_l|x)−logπ_ref(y_w|x)+logπ_ref(y_l|x)，loss=−logσ(βΔ)。在固定 reward 的解析最优解中 β 越大，参考约束更强；但实际 DPO 的有限数据、训练步数和梯度幅度共同影响结果，不能据此给每个实验单调性承诺。

$$\pi^*(y|x) \propto \pi_{ref}(y|x)\,\exp(r(x,y)/\beta)\ ;\ L_{DPO} = -\log\,\sigma(\beta\Delta)$$

实现方案与验证

先核对 chosen/rejected 顺序和 response mask，序列 logprob 按所选目标求和，reference 冻结。初始 actor=reference 时 Δ=0、每对 loss=ln2。再提高 chosen logprob，应使 loss 降低。这个 sanity check 比仅观察训练曲线更直接。

常见追问与易错点

追问：DPO 是不用 reward 吗？答：不用显式训练独立 reward model，但隐式奖励与偏好假设仍在。追问：为什么不用当前 batch 的平均 logprob 代替 reference？答：那改变了目标，不能沿用原推导。

技术依据：[R12] [R14] | 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 17

17 DPO 数据如何构造？为什么 loss 降了，实际效果却变差？

P0 · 优先掌握 公开记录同类主题：[I04] [I09]

> **面试口述版**
>
> 偏好对必须在同一上下文中可比较，标签应代表目标质量而非长度、格式或来源泄漏。DPO loss 降低只能说明训练偏好对更容易区分，不能直接证明在线任务更好。

背景：问题为什么会出现

数据风险包括伪负例过弱、正确答案被误标、两条回答质量差不多、模板泄漏、长度偏置及不同 prompt 被错误配对。SFT 已经让模型偏好 chosen 时，重复优化还可能过度降低 rejected 或损伤覆盖面。

原理与推导：把关键条件说清楚

一个原创诊断：模型学会“长答案优先”，偏好集准确率提升，但短答案事实题下降。若训练集正例长度更长，这种捷径完全可能。应看 chosen/rejected logprob 的独立变化，而不是只看差值；两者都下降，差值也能变大。

实现方案与验证

方案是双人/多评判复核抽样、按难度/长度/任务切片、加入有辨识力的 hard negatives，并对同 prompt 去重和数据泄漏检测。做 SFT only、SFT+DPO、不同 β/epoch 的受控对照；在线评估固定生成预算及 judge 设置。

常见追问与易错点

追问：DPO 不需要采样所以没有分布偏移吗？答：离线偏好集和当前策略输出也会失配。追问：label smoothing 能解决脏数据吗？答：可缓和噪声影响但不替代清洗，也不能保证修复系统偏差。

技术依据：[R12] [R14] [R15] | 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 18

18 ORM、PRM、规则奖励与 reward hacking 如何设计与排查?

P0 · 优先掌握 公开记录同类主题：[I01] [I08]

> **面试口述版**
>
> ORM 给最终结果打分，PRM 对中间步骤提供反馈，规则奖励依赖可执行判据。更密的奖励不自动更可信：若过程判分有偏，模型可能学会讨好评判器。奖励设计必须与独立评估分离。

背景：问题为什么会出现

代码通过测试、数学等价校验、格式合法性、工具任务成功率是不同层次信号。格式奖励只能辅助，不能掩盖正确性失败。多奖励加权还会引入量纲和冲突：高权重长度奖励可能诱导冗长。

原理与推导：把关键条件说清楚

reward hacking 的典型表现是训练 reward 上升、独立成功率不变或下降。先抽样审阅最高分与最低分轨迹，检查是否利用判题漏洞、伪造工具结果、输出测试特判内容，或重复语句骗 judge。环境提供的 tool observation 必须与模型自报结果区分。

实现方案与验证

把训练 verifier 和留出评估器隔离；代码在受限资源的 sandbox 执行；加入隐藏测试与超时/异常分类。对奖励项做相关性与消融，记录 reward 版本；比较同权重不同 checkpoint 在相同固定集上的结果。安全约束不是让模型可自行覆盖的 reward 文本。

常见追问与易错点

追问：给每一步奖励一定解决信用分配吗？答：减少延迟可能有益，但错误 PRM 会错误分配信用。
追问：不可验证的开放任务怎么办？答：采用可靠偏好/人评和多维指标，并诚实说明评判不确定性。

技术依据：[R15] [R16] [R17] | 口述、数值例子及实验建议为本册组织与推导。

02 后训练算法 / QUESTION 19

19 DAPO、Dr. GRPO、GSPO 应怎样比较，而不是只背缩写？

P1 · 进阶补充 公开记录同类主题：[I02] [I11]

> **面试口述版**
> 比较四个位置：advantage 怎么构造、importance ratio 在 token 还是 sequence 层、clip 怎么做、loss 如何归一化。它们是围绕不同训练病症提出的配方，不能只说“都是 GRPO 的升级版”。

背景：问题为什么会出现

DAPO 强调非对称 clipping、动态采样、token-level loss 与长度相关奖励处理；Dr.GRPO 研究长度/难度偏置并调整归一化；GSPO 把重要性权重改到长度归一化的 sequence likelihood，再做序列级 clipping。这里概括原论文，不代表所有框架同名实现都一致。

原理与推导：把关键条件说清楚

GSPO 的一个核心量 s_i=exp[(1/L_i)Σ_t(logπθ−logπold)]，即逐 token ratio 的几何平均。它不是原始序列概率比 Π_t r_t，不能直接套用完整轨迹 IS 的无偏性。不同配方还可能改变 KL、采样过滤和有效训练 token 数。

```
s_i = exp[(1/L_i) Σ_t log(πθ(y_it|s_it)/πold(y_it|s_it))]
```

实现方案与验证

落地对比用同一基模/数据/verifier，记录总生成 token、有效更新 token、wall time、pass@1 和长度分布。先单变量替换归一化，再比较完整配方，避免把额外采样或更宽预算误报成算法本身提高样本效率。

常见追问与易错点

追问：GSPO 稳定 MoE 是否意味着无需 router replay？答：不能从特定论文实验推广为所有系统保证；算法降低某类噪声与内核/路由一致性是两个维度。实现版本、精度、路由机制仍需核验。

技术依据：[R17] [R18] [R19] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 20

20 π_current、π_old、π_rollout、π_ref 四个策略如何区分?

P0 · 优先掌握 公开记录同类主题：[I02] [I13] [I14]

> **面试口述版**
>
> current 是正在更新的 trainer；old 是本轮近端优化基准；rollout 是实际生成数据的行为策略；ref 是约束漂移的参考策略。权重相同不保证 rollout 与 trainer 的数值分布完全相同，推理内核、量化和采样变换都会影响它们。

背景：问题为什么会出现

理想同步 PPO 可把 old 与 rollout 近似视为同一策略；异步系统中 old 可能是较新的训练快照，rollout 数据来自多个旧版本。reference 通常更长期固定，不随每个 mini-batch 刷新。

原理与推导：把关键条件说清楚

π_current/π_old 衡量本轮优化位移；π_old/π_rollout 则刻画行为数据与近端基准的失配。二者数值相乘等于 current/rollout，但在一个目标里把两项分别 stop-gradient、clip 或 rejection 后，不再等价于任意合并后的算法。

$$\pi_{current}/\pi_{rollout} = (\pi_{current}/\pi_{old}) \cdot (\pi_{old}/\pi_{rollout})$$

实现方案与验证

数据 schema 明确存 policy_version、old_logprob 的来源、rollout_logprob 是否含 temperature/top-p、ref 版本。训练前先检查 ratio 的分布；同权重对照同样的 token prefix，比比较两边自由采样输出是否一样更可诊断。

常见追问与易错点

追问：old_logprob 每个 mini-batch 都重算成 current 可否？答：会让 ratio 退回 1，破坏本轮近端基准。追问：ref 能直接作分母吗？答：那是另一种目标，不是通常 PPO 的 old-policy ratio。

分布	职责	典型风险
πrollout	真正生成样本的行为策略	采样配置 / 版本必须记录
πold	本轮 PPO 的固定更新基准	不能随 mini-batch 随意刷新
πcurrent	正在反向更新的策略	形成目标概率与 surrogate
πref	跨轮约束参考分布	不是 behavior 的同义词

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R20] [R21] [R23] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 21

21 训练与 rollout 的 logprob 对不上，从哪里查?

P0 · 优先掌握 公开记录同类主题：[I13] [I14]

> **面试口述版**
>
> 先对齐输入与概率语义，再定位数值和模型执行。优先检查 tokenization、chat template、位置与 mask、采样变换、权重版本；这些正确后，再看低精度、内核归约和 MoE 路由差异，不能一开始就把所有偏差归咎于 FP8。

背景：问题为什么会出现

在一次 RL 更新中，ratio 的指数会放大 logprob 差异。分布失配既可能来自旧数据，也可能在相同权重下出现；自由生成的 token 分岔后比较整段文本无法定位第一处误差。

原理与推导：把关键条件说清楚

建议分层：第一层输入 token/position/attention mask 完全相同；第二层同 checkpoint、同 dtype、关闭 dropout；第三层同前缀做 teacher-forced logprob，比较 max/mean error 与尾部分位数；第四层逐层定位 logits、norm、attention、router 的偏差。最后再开启量化、CUDA Graph 和实际采样配置。

实现方案与验证

用少量固定 prompt 做 golden set，保存每步 sampled-token logprob 和若干 top logits。报告差异分布而非只看平均。若生成器返回的是未过滤 logprob，而真正采样用了 top-p，必须明确哪一个是 behavior 概率。部署任何修正前先做梯度稳定与最终任务指标对照。

常见追问与易错点

追问：bitwise 不同一定有害吗？答：不是，关键是偏差是否被 ratio 放大并影响训练；但平均很小也可能掩盖极端样本。追问：相同随机种子为何不一样？答：随机数消耗顺序、batch 形状和内核数值都可能不同。

技术依据：[R20] [R21] [T19] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 22

22 MoE Router Replay 解决什么？为什么不等于训练完全确定性？

P1 · 进阶补充 公开记录同类主题：[I13] [I14]

> **面试口述版**
>
> Router Replay 把某次路由选择记录下来，在后续对应计算中重用，避免 token 因微小数值扰动换专家。它约束的是离散路由路径，不会自动统一专家 GEMM 的数值、量化 scale、通信归约、采样或优化器状态。

背景：问题为什么会出现

MoE Top-k 在边界附近不连续：两个专家打分几乎相等时，微小误差就会改变选择。对 RL 的 logprob ratio，这种分支变化可能大于普通浮点误差。激活重计算、训练/推理不同后端，都可能引入路由重选。

原理与推导：把关键条件说清楚

要区分训练前向与重计算/反向的一致性，以及 rollout 到训练的一致性。不同项目用 R2/R3 等名字描述不同覆盖范围，面试时应按“记录者、重放者、保存哪些字段”定义，不靠缩写。只保存专家 ID 与保存权重/排序/位置映射也不是相同语义。

实现方案与验证

检查 token reorder、padding、TP/EP rank 映射、layer ID、sequence ID 在跨引擎传递时不被破坏。用固定路由对比 ref logits、梯度、吞吐与训练曲线；还要测路由 metadata 显存和传输成本。不能只凭 forward output close 就宣称长期 RL 稳定。

常见追问与易错点

追问：固定路由会不会阻止 router 学习？答：取决于重放范围和梯度定义；固定离散选择与是否更新连续 routing weights 是两件事。应明确哪些 forward 复用，何时生成新路由，而非永久冻结整个 router。

技术依据：[R21] [R20] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 23

23 异步 RL 为什么更快，又为什么更容易 off-policy?

P0 · 优先掌握 公开记录同类主题：[I13] [I14]

> **面试口述版**
>
> 同步训练等待整批最长轨迹完成；异步把生成与训练解耦，减少长尾等待。但训练开始时数据可能来自旧权重，生成端还可能在同一轨迹中跨版本。吞吐收益要与样本陈旧、重要性修正和最终质量一起评估。

背景：问题为什么会出现

简单系统由 rollout workers、轨迹队列、reward workers、trainer 和权重发布器组成。生产速度大于消费速度时，队列增长会把数据变旧；反过来则 trainer 空转。队列不是越大越好，GPU 利用率高也不代表有效学习更快。

原理与推导：把关键条件说清楚

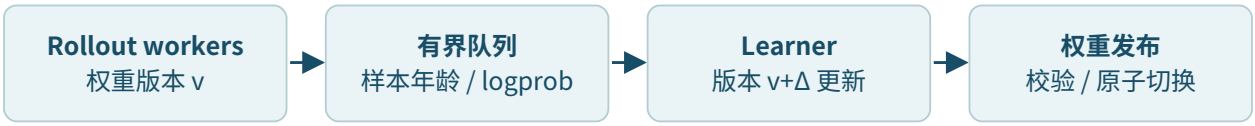
原创例子：同步每批等待 90 秒生成和 30 秒训练，若能理想解耦，稳态周期下界是 max(90,30)=90 秒，而非 120 秒；但实际还包含同步、评分、排队和样本淘汰，不能直接把该下界当实测加速。

实现方案与验证

按版本差或时间定义 staleness，设置队列上限、背压、过期数据丢弃与消费配额。记录每条 trajectory 的 behavior version；若允许中途换权重，就需要更细粒度 provenance 与算法支持。先从同步 correctness baseline 开始，再逐步增加异步深度。

常见追问与易错点

追问：IS 能完全解决 staleness 吗？答：支持集、状态分布、长程连乘方差仍存在；clip/truncation 还引入偏差。追问：比较用 tokens/s 足够吗？答：更应比较达到同一 held-out 分数的 wall time 与 GPU-hours。



吞吐提升必须同时报告策略滞后、有效样本率和达到目标质量的时间。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R23] [R20] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 24

24 Importance correction、rejection sampling 与 PPO clipping 能否混用?

P1 · 进阶补充 公开记录同类主题：[I02] [I13]

> **面试口述版**
>
> 可以按明确目标组合，但它们解决的层次不同：IS 校正行为分布，rejection 丢弃过度失配数据，PPO clip 限制本轮策略更新激励。把三个阈值都调紧，可能只剩极少有效样本。

背景：问题为什么会出现

先指定被估计的目标和样本分布，再定义修正权重。截断 IS 限制方差，却有偏；self-normalized IS 也通常是有限样本有偏。rejection 后的训练分布受到接受条件改变，报告原始样本数不足以描述实际更新。

原理与推导：把关键条件说清楚

$ESS=(\Sigma w)^2/\Sigma w^2$。举例权重 [1,1,1,1] 时 ESS=4；[100,1,1,1] 时 ESS≈1.06，说明几乎一个样本主导。若直接 cap 为 2，会改善权重均衡，却改变原始期望。序列 ratio 取几何平均也不是对原乘积的无损数值技巧。

实现方案与验证

同时画原始 logratio 分位数、截断比例、接受率、ESS、有效 group 数和最终成功率。先比较“无失配同步基线→有失配未修正→有修正”，否则可能用复杂修正掩盖 tokenizer 或旧权重的实际 bug。

常见追问与易错点

追问：batch normalize 权重是否总有益？答：可能稳定尺度，但改变估计器，必须记录。追问：全部拒绝怎么办？答：当前批次不更新或重新采样并报警，不能默默分母 clamp 后当作正常一次学习。

技术依据：[R20] [R23] [R03] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 25

25 训练权重如何同步到 vLLM/SGLang？重分片与 KV cache 怎么处理？

P0 · 优先掌握 公开记录同类主题：[I13] [I14]

> **面试口述版**
>
> 权重同步不是简单 broadcast 一个文件：训练可能是 FSDP/TP/PP 分片，rollout 可能用不同 TP 与布局，需要按参数语义重分片、传输并原子切换版本。旧 KV cache 由旧权重产生，通常不能在新权重下无条件继续使用。

背景：问题为什么会出现

参数名、QKV 合并布局、专家分布、quantized weight 与 scale 必须一起转换。一次完整发布应保证消费者不看见半新半旧参数，否则同一 forward 已不是某个明确版本的模型。

原理与推导：把关键条件说清楚

可选路径包括先 gather 再 scatter、直接 shard-to-shard、分层节点内/节点间传输。前者易实现但产生显存峰值，后者复杂却可能减少中转。同步字节量可从参数 dtype 和实际复制路径估算，不能只用文件大小忽略 scale、padding 和临时副本。

实现方案与验证

采用版本号、参数清单与 hash、加载完成 barrier、短 canary logprob 对照，再允许 rollout 使用。对于进行中的请求，要么固定版本跑完，要么受控中断并正确重建 cache；简单修改 model weights 后复用旧 cache 是高风险路径。

常见追问与易错点

追问：为什么只同步 LoRA 更快？答：变化的参数较少，但仍要保证 base、adapter、merge/unmerge 与版本一致。追问：load 完后 logits close 就够吗？答：还需多层/多专家覆盖、量化 scale 和长上下文验证。

技术依据：[R22] [R24] [T22] ｜ 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 26

26 verl、Ray、FSDP/Megatron、rollout engine 分别负责哪层？

P0 · 优先掌握 公开记录同类主题：[I01] [I13]

> **面试口述版**
>
> verl 等框架组织 RL 数据流与角色；Ray 常用于资源/worker 与远程任务编排；FSDP/Megatron 执行分布式训练；vLLM/SGLang 一类引擎执行高效生成。分布式编排不等于数学归约正确，生成快也不等于整轮 RL 快。

背景：问题为什么会出现

一条轨迹从 dataset 进入 rollout，再经过 reward、logprob/value、advantage 与更新。每条边都是实际数据依赖：token、mask、old/ref logprob、路由 metadata 等，任意字段 shape 或版本不一致都可能引发静默错误。

原理与推导：把关键条件说清楚

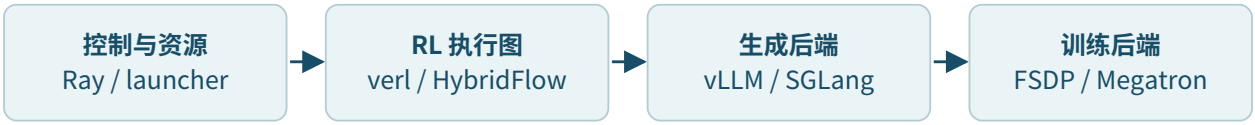
HybridFlow 的关键思路是区分角色间数据流和角色内多 GPU 计算。为任务选择 backend 时，要比较支持的并行、模型结构、内存策略和状态同步成本，不能仅以“某框架支持 GRPO”推断它支持你的 MoE、精度和 Agent loop 组合。

实现方案与验证

排查按层走：控制面确认 worker placement 和生命周期；数据面确认 batch schema 与序列化；算力面分析训练/生成 kernel；通信面看 collective 与权重刷新；算法面核对 loss/advantage。最小复现实验先缩为单角色单批次，再逐个接回组件。

常见追问与易错点

追问：Ray 调度慢和 NCCL 慢怎么区分？答：看任务提交/等待的 CPU 时间与 GPU stream 事件；一个是控制路径，一个是设备通信路径。追问：worker 存活就代表进度正常吗？答：还要有心跳之外的 batch/版本进度指标。



框架是执行组织方式，不替代 PPO / GRPO 的数学定义。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R22] [R24] [T10] [T11] ｜ 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 27

27 训练与 rollout 同卡部署还是分离部署？怎样做资源决策?

P0 · 优先掌握 公开记录同类主题：[I13] [I14]

> **面试口述版**
>
> 同卡部署减少长期重复权重，适合阶段性复用 GPU，但要支付切换、offload/reload 或重分片成本；分离部署更容易并行执行生成与训练，却增加权重副本、同步和 staleness 管理。没有不看 workload 的固定赢家。

背景：问题为什么会出现

先测每阶段 GPU 时间、峰值显存和同步量，再分配资源。长回答、复杂工具环境可能让 rollout 与 reward 主导；大 batch 更新则可能让 trainer 主导。总卡数相同的比较还要计入闲置、缓存与环境 CPU 开销。

原理与推导：把关键条件说清楚

用 producer-consumer 近似：若每条轨迹平均消耗生成 token g、训练 token t，可分别测两侧吞吐 q_r 与 q_t，目标是 q_r/g≈q_t/t 的有效轨迹生产消费速率。实际还受 group 完整性、长尾与过滤率约束。

实现方案与验证

建议先建立可复现的同卡同步基线，再尝试不同卡数配比的分离部署。每个方案报告 GPU-hours/有效样本、固定评测阈值达成时间、同步时间与样本年龄。不要把卸载到 CPU 的等待从 step time 中排除。

常见追问与易错点

追问：显存刚好装下就可以吗？答：还需 KV 峰值、通信 buffer、CUDA Graph 池和权重发布临时副本余量。追问：rollout 卡越多越好？答：消费侧不变会堆积陈旧数据，可能降低有效样本效率。

技术依据：[R22] [R23] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 28

28 Rollout 吞吐怎么优化？长尾、KV cache、group 与并发的权衡？

P0 · 优先掌握 公开记录同类主题：[I01] [I13]

> **面试口述版**
>
> 先分解 prefill、decode、环境等待、评分和调度，再优化主导项。连续批处理与动态 token budget 可提升设备利用，但 group 等待和长尾响应会影响 RL 更新节奏；限制长度也会改变任务分布与奖励。

背景：问题为什么会出现

生成吞吐不是输入样本数除时间。一个 prompt 的 G 个候选可能长度差异很大；某些候选等待工具，另一些已结束。同步按组更新时，短候选完成并不意味着对应 GPU/训练数据立即可用。

原理与推导：把关键条件说清楚

KV 近似字节量为 2 · layers · live_tokens · kv_heads · head_dim · bytes，GQA/MQA 主要减少 kv_heads。它不等于模型参数显存。beam、前缀复用、分页碎片与多模态部分会改变实际值，因此用公式估算后必须看实测峰值。

$$\text{KV bytes} \approx 2 \times L_{layers} \times T_{live} \times H_{KV} \times D_{head} \times b$$

实现方案与验证

记录 time-to-first-token、decode tokens/s、活跃序列数、prefill queue、KV 占用和超时率。按 token 数而非固定样本数做 batch，隔离慢工具，设置最大在途轨迹。停止条件、EOS、max_new_tokens 与 reward 对截断的处理应一并版本化。

常见追问与易错点

追问：只统计输出 tokens/s 是否够？答：还要看正确有效样本/s，以及为了筛掉零方差组额外生成了多少。追问：prefix cache 一定命中有益吗？答：权重版本变更和不同采样上下文会限制安全复用。

技术依据：[R22] [R23] [R24] | 口述、数值例子及实验建议为本册组织与推导。

03 RL Infra 与一致性 / QUESTION 29

29 Agentic RL 如何做多轮信用分配？工具结果为什么要 mask?

P0 · 优先掌握 公开记录同类主题：[I01] [I08] [I12]

> **面试口述版**
>
> Agent 的轨迹包含模型动作与外部观察。训练 actor 应对自己选择的文本/工具调用求 logprob，工具返回结果通常作为条件输入而非被学习的动作；最终任务奖励需要向先前动作分配信用，不能把外部结果当成模型生成能力。

背景：问题为什么会出现

例如“搜索→阅读→计算→回答”，最终正确可能取决于第一次搜索参数。只训练最后一句回答会丢失早期动作信号；把工具响应也加进 loss 则错误地要求 actor 生成环境事实。

原理与推导：把关键条件说清楚

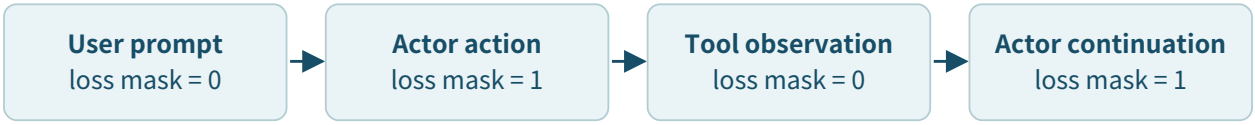
可以按 token 时间或 turn 时间建模，但折扣、奖励位置、mask 必须与粒度一致。工具失败可能是服务故障，也可能是模型参数错误；把两者都给同样负奖励会混淆可控性。步骤奖励需有可靠判据，不能只按步骤数量鼓励无意义调用。

实现方案与验证

为每轮记录 action span、observation span、tool name/status、latency、environment version 与终止原因。将 final reward、过程反馈、成本惩罚分开存；验证只有 action span 对 actor 产生梯度。对异常服务重试、缓存与失败标签做固定策略，避免数据选择偏差。

常见追问与易错点

追问：工具结果不计 loss，那后续输出能看见它吗？答：能，mask 只控制损失项，不等于从 attention 上下文删除。追问：时间限制如何处理？答：取决于任务是否视为失败终止，不能一律用 done=1。



mask = 0 不等于不可见：工具观察仍影响后续 actor 的条件概率。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R24] [R05] [R16] | 口述、数值例子及实验建议为本册组织与推导。

04 诊断、评估与实验 / QUESTION 30

30 奖励上涨但成功率下降，entropy collapse 或 KL 爆炸如何排查？

P0 · 优先掌握 公开记录同类主题：[I08] [I11]

> **面试口述版**
>
> 先确认数据和评估口径没有变化，再检查奖励是否被利用、loss/mask 是否正确以及策略更新是否过猛。reward、entropy、KL、clip fraction 各描述一个侧面，单个指标不能判定学习正常。

背景：问题为什么会出现

建立分层仪表盘：任务层成功率/错误类型；数据层难度、长度、零方差组；算法层 reward 分解、advantage、entropy、ratio；系统层版本滞后、吞吐、OOM 与重试。很多“算法崩了”实际是 reward 或旧权重管线出错。

原理与推导：把关键条件说清楚

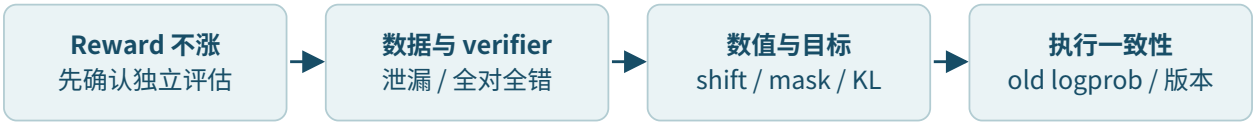
若 reward 与独立 eval 背离，先看奖励漏洞与验证集污染；若 logratio 尾部突然变大，检查版本、采样配置和 MoE mismatch；若 entropy 快速接近零并重复回答，检查学习率、更新轮数、奖励可区分性和 clip 设置；若 loss 为零，检查全部 mask 或组方差为零。

实现方案与验证

保存出问题前后的 checkpoint、固定 batch、随机种子与配置。先在固定 batch 重放一到数步，观察 logits/grad 的首次异常；然后恢复正常数据流。按单因素做 rollback，不同时修改 reward、温度、β 和学习率。所有修复都要回到独立任务集验证。

常见追问与易错点

追问：直接把 KL 系数调大能解决吗？答：若问题是标签错位或行为概率不对，调 β 只掩盖故障。追问：所有指标平稳就没问题？答：可能没有有效梯度或任务太简单，仍须看可区分的留出评估。



按最小可复现案例逐层排除，不同时更改算法与系统多个变量。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R16] [R17] [R20] [R23] | 口述、数值例子及实验建议为本册组织与推导。

04 诊断、评估与实验 / QUESTION 31

31 RL 的显存账本与 OOM，为什么不能只算 actor 参数?

P0 · 优先掌握 公开记录同类主题：[I02] [I13]

> **面试口述版**
>
> 按阶段列出 actor/critic 的训练状态、reference/reward 的只读权重、rollout 的 KV、激活、logits、logprob、通信与临时重分片 buffer。常见 OOM 发生在阶段切换或权重刷新峰值，而非稳定训练时。

背景：问题为什么会出现

采用一个明确假设：BF16 参数 2B、BF16 梯度 2B、FP32 master 4B、两个 FP32 Adam 状态 8B，共 16 bytes/训练参数；冻结模型可能只有权重和前向工作区。BF16 训练并非始终有 master copy，实际框架要检查。

原理与推导：把关键条件说清楚

若 actor 7B，在未分片的该假设下仅模型训练状态就是 112 GB（十进制），还没算激活。GRPO 去掉同尺寸 critic 可省大量训练状态，但增大 G 会增加并发生成、KV 和候选缓存，所以端到端峰值不一定同比下降。

实现方案与验证

用阶段标记记录 allocated/reserved/峰值；先单独跑 rollout 和 update，再连接权重同步。对 KV 并发、训练 microbatch、activation checkpoint、offload、参数分片分别做预算。中间 logits 只保留必要位置或分块计算，但必须保持 loss 数学一致。

常见追问与易错点

追问：empty_cache 能解决持有计算图的 OOM 吗？答：不能，仍被引用的张量不会释放。追问：全量 logprob 是否等于全词表 logits？答：不是，采样 token logprob 通常远小；训练 CE/logsoftmax 的临时张量仍可能很大。

技术依据：[R22] [T08] [T19] | 口述、数值例子及实验建议为本册组织与推导。

04 诊断、评估与实验 / QUESTION 32

32 分布式 RL 的 loss、advantage、gradient 应如何归约?

P0 · 优先掌握 公开记录同类主题：[I14] [I13]

> **面试口述版**
>
> 先确定优化的是全局 token 均值还是每条轨迹均值。DDP 默认对梯度做平均，但这不自动修正各 rank 不同的有效 token 数；日志上的 all_reduce 也不等价于正确反传。group advantage 必须在正确 prompt 分组内计算。

背景：问题为什么会出现

令 rank r 的有效 token 损失和为 S_r，计数为 N_r，目标 L=ΣS_r/ΣN_r。若 W 个 DP rank 最后平均梯度，每个 rank 应使用 W · S_r/N_global（在本例条件下）反传，这样平均后才得到全局目标梯度。直接用 S_r/N_r 会过度加权短 rank。

原理与推导：把关键条件说清楚

原创例子：rank0 有 1 token、损失 2；rank1 有 3 tokens、损失和 12。平均本地均值是 (2+4)/2=3，全局均值是 14/4=3.5。差异会进一步传到梯度，不是打印数字的美观问题。

```
L_global = (Σ_r S_r)/(Σ_r N_r) ; L_local,r = W·S_r/N_global
```

实现方案与验证

归约有效计数而非 padding 长度；整个 accumulation window 使用同一个目标分母。明确通信后端的梯度 sum/mean 语义，FSDP 或自定义 hook 不要盲套 W。测试把同一逻辑 batch 拆成不同 rank/microbatch 后，梯度应与单卡参考相符。

常见追问与易错点

追问：advantage whitening 要跨所有 rank 吗？答：取决于配方；跨 batch whitening 和同 prompt group 标准化含义不同。追问：零有效 token rank 怎么办？答：需要保持 collective 顺序并安全地产生零贡献，不能本地直接跳过必要通信。

技术依据：[T09] [R13] [R26] | 口述、数值例子及实验建议为本册组织与推导。

04 诊断、评估与实验 / QUESTION 33

33 RL checkpoint 应保存哪些状态？可重复性怎样分层验收？

P1 · 进阶补充 公开记录同类主题：[I13] [I14]

> **面试口述版**
>
> 除了模型和优化器，还要保存 scheduler、RNG、数据游标、奖励与采样配置、policy version、reference 标识，以及异步系统队列/在途轨迹的恢复策略。仅保存 actor 权重能恢复推理，不一定能续训到相同状态。

背景：问题为什么会出现

同步 checkpoint 容易定义一个一致更新边界；异步系统还要处理 rollout 已生成、已评分、已消费之间的状态。恢复时盲目重放在途数据，可能重复更新；全部丢弃则改变样本分布和进度。

原理与推导：把关键条件说清楚

建议定义三个验收级别：数值单元算子 close；固定 batch 与配置下短程梯度/指标可复现；较长训练的统计性能在误差范围内一致。跨硬件、内核、world size 的 bitwise 一致通常不能仅凭设 seed 宣称。

实现方案与验证

采用事务式 checkpoint manifest：写临时分片，完成所有角色快照与版本记录，再发布完成标志。恢复测试必须真的中断进程并重启；对异步队列使用确定的保留/丢弃规则和唯一 trajectory ID，避免重复消费。

常见追问与易错点

追问：改变 rollout 并发数为什么结果变了？答：采样 RNG 消耗、batch 数值与队列次序都可能改变。追问：路由重放能替代 RNG 保存吗？答：不能，它只覆盖一类离散执行路径。

技术依据：[T22] [T23] [R23] ｜ 口述、数值例子及实验建议为本册组织与推导。

04 诊断、评估与实验 / QUESTION 34

34 pass@1、pass@k、best-of-N 和训练 reward 应怎样比较?

P0 · 优先掌握 公开记录同类主题：[I01] [I08]

> **面试口述版**
>
> pass@1 衡量一次采样成功率；pass@k 关注 k 次中至少一次成功；best-of-N 还需要实际可用的选择器。预算、温度、样本相关性和 verifier 都影响结果，不能拿更大的 k 直接证明模型本身更强。

背景：问题为什么会出现

若某题独立采样 n 次有 c 次成功，经典无偏估计为 1−C(n−c,k)/C(n,k)，需要 n⩾k。它是对抽取 k 个候选的成功概率估计，不等同于“真实线上每次都能选出正确答案”。

原理与推导：把关键条件说清楚

原创例子 n=5、c=2、k=2，pass@2=1−C(3,2)/C(5,2)=0.7。即便至少一个正确，选择器若只看长度，也可能选错。报告时应分开 oracle pass@k 与实际 verifier/reranker 后的成功率。

$$\text{pass@k} = 1 - C(n-c,k)/C(n,k) \ (n \geq k)$$

实现方案与验证

评测固定 prompt、最大 tokens、temperature、tool budget 和时间限制。按题聚合并做 bootstrap 置信区间，避免把同一题多条样本当完全独立测试题。隔离训练 verifier 与 holdout，用去重/污染检查和失败案例支持结论。

常见追问与易错点

追问：RL 后长回答更多导致分数提高，算不算提升？答：是特定预算下结果，但应另外给固定 token/时延预算对照。追问：训练 reward 可直接作验收？答：不能替代独立指标，否则可能只在优化评判器。

技术依据：[R25] [R16] | 口述、数值例子及实验建议为本册组织与推导。

04 诊断、评估与实验 / QUESTION 35

35 怎样设计 RL 消融实验，避免把系统加速说成算法提升？

P0 · 优先掌握 公开记录同类主题：[I01] [I11]

> **面试口述版**
>
> 将算法效果、样本效率和系统吞吐分开评估。固定生成 token 预算比较学习效率，固定 GPU-hours 比较成本效果，固定任务分数比较 time-to-target；三种指标回答的是不同问题。

背景：问题为什么会出现

一次修改可能同时改变 loss、采样过滤和并发。比如动态采样保留更有效 group，update tokens/s 提高，却可能先消耗了更多 rollout；只展示 trainer 利用率会遗漏主要成本。

原理与推导：把关键条件说清楚

建议实验矩阵：基线；只改 advantage；只改 loss reduction；只改 rollout correction；完整组合。每组记录相同基模、数据 split、种子集合、reward 版本与总采样量。先做小规模筛选，再对候选用多种子和更长训练验证。

实现方案与验证

用学习曲线而非只看最后一个最好 checkpoint；选模策略预先规定。对于系统优化，确保输出分布和训练目标不变或明确标出变化；比较覆盖初始化、权重同步、评分、重试与 checkpoint 的端到端时间。

常见追问与易错点

追问：只能跑一个 seed 怎么办？答：如实标为单次结果，保留曲线、误差来源与待重复项，不报统计显著。追问：减少输出长度提高吞吐是不是纯系统优化？答：通常也改变任务行为与预算，需单列。

技术依据：[R17] [R18] [R23] | 口述、数值例子及实验建议为本册组织与推导。

04 诊断、评估与实验 / QUESTION 36

36 RL 的 batch、group、mini-batch、micro-batch 与 epoch 怎样调?

P0 · 优先掌握 公开记录同类主题：[I02] [I11] [I13]

> **面试口述版**
>
> 先统一单位：prompt batch 是问题数，group size 是每题生成条数，update mini-batch 决定一次优化使用的数据，micro-batch 主要用于内存切分；PPO epoch 是同一批轨迹的复用次数。不同框架字段名相同也可能单位不同。

背景：问题为什么会出现

教学例子：128 个 prompt、G=8，最多得到 1024 条候选；过滤后有效条数可能更少。训练 mini-batch=256 条、micro-batch=8 条时，梯度累积次数还取决于 DP 划分。若按 token budget 动态批处理，固定“条数”公式必须改写。

原理与推导：把关键条件说清楚

增大 G 影响探索、组内统计、生成成本与 KV；增大训练 batch 影响梯度噪声；多 epoch 提高复用却增加对 old policy 的偏离；减小 micro-batch 理论上可保持目标不变，但归一化、dropout 和数值顺序可能带来差异。不要把所有 batch 参数一并等比缩放。

实现方案与验证

先固定学习率和 loss 定义，检查 micro-batch 拆分后的梯度等价，再扫 G 与有效 prompt 数；随后调整学习率、epoch、clip 与 KL。每次只改一到两个相关参数，追踪 reward、长度、ratio、ESS 和 GPU-hours，而不以单次最高分选型。

常见追问与易错点

追问：显存不够能只减 mini-batch 吗？答：这会改变优化统计；优先评估 micro-batch 与 checkpoint 等保持目标的手段。追问：扩大 batch 必须线性增大学习率吗？答：是可检验启发式，不是 PPO/GRPO 的普遍定律。

技术依据：[R26] [R13] [R03] | 口述、数值例子及实验建议为本册组织与推导。

05 手写、设计与项目 / QUESTION 37

37 手写题：实现带 mask 的 PPO loss，要有哪些单元测试?

P0 · 优先掌握 公开记录同类主题：[I02] [I13]

> **面试口述版**
>
> 输入 new_logprob、old_logprob、advantage 和有效 token mask；在 log 空间求差后指数得到 ratio，计算两条 surrogate 的较小值，再按指定有效 token 分母求负均值。old 与 advantage 不反传，mask 不能只在最后打印时使用。

背景：问题为什么会出现

代码核心见附录 A；这里只实现单进程、token mean 的基本 actor loss，不夹带 value/KL，也不伪装成完整 PPO trainer。无有效 token 的 batch 应显式报错或由上层按协议处理，不能静默产生一个“看似合理的零损失”。

原理与推导：把关键条件说清楚

四类测试：new=old 时 ratio=1；A>0/r>上界时局部梯度被 clip；A<0/r<下界时同理；反方向仍有梯度。再测 padding 梯度为零、old 不收梯度、极端 logratio 的有限性，以及两种分批方式归约是否一致。

实现方案与验证

先用 float64 CPU 参考小张量，核对 autograd；生产 FP32/BF16 版本用合理误差比较。loss 的 sum 与 mean、group mask 和 sequence mask 不应隐藏在深层 helper 中，面试现场要能说清它们改变哪个目标。

常见追问与易错点

追问：把 old_logprob 直接设成 new_logprob.detach() 可以吗？答：在每次调用重新这么做会破坏固定 old 基准；只是初次同权重 sanity check 允许。追问：如何防 exp 溢出？答：先检测失配来源，必要时选择有明确算法含义的限制，不能任意 clamp 后宣称同一目标。

技术依据：[R03] [R04] [T05] | 口述、数值例子及实验建议为本册组织与推导。

05 手写、设计与项目 / QUESTION 38

38 手写题：GRPO advantage 与 DPO loss，怎么验证符号和分组?

P0 · 优先掌握 公开记录同类主题：[I04] [I09] [I12]

> **面试口述版**
>
> GRPO 以同 prompt 的 G 条 reward 为一组做中心化/可选标准化；DPO 对同 prompt 的 chosen/rejected 计算相对 reference 的 logprob margin，再用 −logsigmoid。两者不能把 batch 任意 reshape 后就认定组对齐。

背景：问题为什么会出现

附录 B 给出两个纯函数。GRPO 演示采用总体标准差 correction=0，并明确 ε 与零方差处理；实际库若采用样本标准差，数值会不同但不一定是 bug，应以配方为准。DPO 演示输入已正确求和的 response sequence logprob。

原理与推导：把关键条件说清楚

测试 GRPO 奖励 [0,0,1,1] 应得到 [-1,-1,1,1]（忽略 ε）；常数组输出零；组内调换样本后输出应同样调换。DPO 在 actor=reference 时 loss=ln2；提升 chosen 相对 rejected 的 margin 应降低 loss；正负交换应改变方向。

实现方案与验证

真实数据管线再测试 prompt ID、answer ID、response span 与 mask。若把 prompt tokens 加入 pair logprob，相同 prompt 部分在理想完全相同条件下可消去，但 truncation、模板或拼接差异会破坏这一点；规范做法仍是显式定义目标 span。

常见追问与易错点

追问：group mean 为零就证明实现正确吗？答：混了不同 prompt 也可能均值为零，必须验证分组语义。追问：DPO 的 rejected 是否必须错误？答：是相对不优，不必绝对错误，标签需符合目标偏好。

技术依据：[R11] [R12] [R14] | 口述、数值例子及实验建议为本册组织与推导。

05 手写、设计与项目 / QUESTION 39

39 系统设计：8 张 GPU 上搭一套可验证的 GRPO 训练链路

P0 · 优先掌握 公开记录同类主题：[I01] [I13]

> **面试口述版**
>
> 先把未知条件显式化：模型大小、卡显存、互联、上下文与回答长度、G、目标训练 batch、奖励成本和时间预算。我的首版目标是正确可恢复，再从 profile 决定同卡复用还是训练/生成分离，不直接承诺某个吞吐数。

背景：问题为什么会出现

教学假设：8 卡、每卡可用预算 80 GiB、7B 稠密 actor、BF16 与 Adam、规则奖励、同机高速互联。这不是指定硬件的规格或实测。未分片 actor 状态约 104.3 GiB；若 8 路完整分片，常驻模型状态粗估 13.0 GiB/卡，但激活、gather、KV 等未计入。

原理与推导：把关键条件说清楚

首版同卡阶段化执行：初始化→采样并记录 behavior→规则评分→group advantage→训练更新→权重发布。限制 rollout live token 与训练 microbatch，逐步增加；若生成占比和长尾突出，再尝试 4+4 分离。4 卡 trainer 分片更粗、rollout 副本更多，不一定总是更快。

实现方案与验证

验收顺序：固定 batch 数值；完整一轮日志；连续多轮梯度/奖励；重启恢复；profiling；长时评估。保留 NaN/OOM/超时的可回放样本，设资源隔离和 checkpoint。只有达到相同任务分数的成本降低，才把方案列为有效优化。

常见追问与易错点

追问：为什么不用最大 G？答：增加采样成本与 KV 压力；需要看有效梯度样本密度。追问：怎么给结论？答：提供配置、版本、预算账本和实测曲线；未测的性能不给数字。

技术依据：[R22] [R23] [T08] | 口述、数值例子及实验建议为本册组织与推导。

05 手写、设计与项目 / QUESTION 40

40 项目深挖：如何把“我改过 RL 框架”讲成可验证的工程贡献？

P0 · 优先掌握 公开记录同类主题：[I01] [I07] [I12]

> **面试口述版**
> 按“问题现象→基线与假设→定位证据→最小修改→正确性回归→端到端收益→代价和边界”叙述。面试官要区分你是改配置、维护训练脚本，还是理解算法与数据流后定位了真实问题。

背景：问题为什么会出现

不要只说“加速 30%”或“解决一致性”。先给 workload：模型/精度/并行、输入输出长度、batch、拓扑；再说明原始瓶颈在哪个阶段。若是开源贡献，给 commit、测试、profile 和 issue 关系，不虚构尚未合并的结果。

原理与推导：把关键条件说清楚

以训练/rollout mismatch 为例：先固定 token 前缀发现 logratio 尾部异常，排除 tokenizer/权重版本，再定位某层路由差异；实现路由记录与回放后，对照 logits/gradient、clip fraction、稳定性和吞吐。若只是修复配置错误，应直接说清，不包装成新算法。

实现方案与验证

准备一页“证据卡”：旧/新配置、正确性阈值、分阶段时间、端到端指标、内存代价、未覆盖模型及回滚办法。回答 trade-off 时主动讲更大的 metadata 或较低并发等成本；这种边界比空泛结论更可信。

常见追问与易错点

追问：没有公开代码怎么办？答：用脱敏架构、可公开的等价最小复现和测量口径，不泄露公司参数/数据。追问：无法证明因果怎么办？答：说清相关性与已做消融，列出仍需验证的替代解释。

技术依据：[R20] [R22] [T25] | 口述、数值例子及实验建议为本册组织与推导。

附录 A / CPU REFERENCE

A1. PPO-Clip：损失、mask 与 detach

各页函数共用 import torch 与 import torch.nn.functional as F。下列代码用于检查数学语义，不包含完整 RL trainer、分布式封装或生产级异常恢复。

```
def masked_mean(x: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
    if x.shape != mask.shape:
        raise ValueError("x and mask must have the same shape")
    valid = mask.to(torch.bool)
    count = valid.sum()
    if count.item() == 0:
        raise ValueError("no valid entries")
    return x.masked_select(valid).sum() / count

def ppo_loss(new_logp: torch.Tensor, old_logp: torch.Tensor,
             adv: torch.Tensor, mask: torch.Tensor,
             eps: float = 0.2) -> torch.Tensor:
    if not (new_logp.shape == old_logp.shape == adv.shape == mask.shape):
        raise ValueError("all inputs must have identical shapes")
    if not 0 < eps < 1:
        raise ValueError("eps must be in (0, 1)")
    ratio = (new_logp - old_logp.detach()).exp()
    advantage = adv.detach()
    raw = ratio * advantage
    clipped = ratio.clamp(1 - eps, 1 + eps) * advantage
    return -masked_mean(torch.minimum(raw, clipped), mask)
```

> 函数保留输入浮点 dtype，测试使用 float64；GPU 训练通常应以至少 FP32 计算 logprob 差与 ratio。极端 logratio 需要先诊断，随意 clamp 会改变目标。这里显式检查空 mask，生产环境应避免频繁 .item() 触发主机同步。

附录 A / CPU REFERENCE

A2. GAE：不要用一个 done 代替所有边界

单条未 padding 轨迹参考。真实终止通常 bootstrap=0、continuation=0；外部时间截断可 bootstrap=1、continuation=0。next_value 必须取最终 observation，而不是 reset 后的新状态。

```
def gae(reward: torch.Tensor, value: torch.Tensor,
        next_value: torch.Tensor, bootstrap: torch.Tensor,
        continuation: torch.Tensor, gamma: float = 0.99,
        lam: float = 0.95) -> tuple[torch.Tensor, torch.Tensor]:
    """One unpadded trajectory segment; masks encode boundary semantics."""
    xs = (reward, value, next_value, bootstrap, continuation)
    if reward.ndim != 1 or any(x.shape != reward.shape for x in xs):
        raise ValueError("all inputs must be 1-D with the same length")
    if not (0 <= gamma <= 1 and 0 <= lam <= 1):
        raise ValueError("gamma and lambda must be in [0,1]")
    with torch.no_grad():
        delta = reward + gamma * bootstrap * next_value - value
        result = torch.zeros_like(reward)
        carry = reward.new_zeros(())
        for t in range(reward.numel() - 1, -1, -1):
            carry = delta[t] + gamma * lam * continuation[t] * carry
            result[t] = carry
        return result, result + value
```

> 返回 advantage 与 value target；两者在 no_grad 中构造。多条轨迹要分别处理或提供正确边界 mask。若长度上限本身就是任务定义的终点，应使用任务终止语义。

附录 A / CPU REFERENCE

A3. GRPO 优势与 DPO 目标

下面只实现 GRPO 的组内优势归一化，不是完整 GRPO loss；后者还涉及采样动作概率比、mask、聚合口径与可选 KL。DPO 输入则是回答部分的序列 logprob 之和。

```
def group_advantage(rewards: torch.Tensor,
                    eps: float = 1e-6) -> torch.Tensor:
    if rewards.ndim != 2 or rewards.shape[1] < 2 or eps <= 0:
        raise ValueError("expect [prompts, group>=2], eps>0")
    r = rewards.detach()
    mean = r.mean(dim=1, keepdim=True)
    std = r.std(dim=1, keepdim=True, correction=0)
    return (r - mean) / (std + eps)

def dpo_loss(chosen: torch.Tensor, rejected: torch.Tensor,
             ref_chosen: torch.Tensor, ref_rejected: torch.Tensor,
             beta: float = 0.1) -> torch.Tensor:
    if not (chosen.shape == rejected.shape == ref_chosen.shape
            == ref_rejected.shape) or beta <= 0:
        raise ValueError("inconsistent shape or nonpositive beta")
    # Inputs are response-only sequence logprob SUMS.
    margin = ((chosen - rejected)
              - (ref_chosen - ref_rejected).detach())
    return -F.logsigmoid(beta * margin).mean()
```

> population std（correction=0）与 sample std 并不相同，必须与训练配方一致。DPO chosen/rejected 的 prompt 应相同；reference 项不更新。验证时还需覆盖不同回答长度、tokenizer 模板和截断。

附录 B / ACTUAL CHECKS

实际 CPU 验证结果

执行环境：CPU，PyTorch 2.10.0+cpu，float64。检查日期 2026-10-31。本册相关检查共 7 组、46 个测试情形；“情形”不是独立实验或性能样本。

检查组 / 情形数	最大绝对误差	验证内容与限制
PPO 四分支数值与梯度 4 情形 · 通过	0.000e+00	正优势上侧/负优势下侧为平坦分支；反方向梯度保留；old 与 advantage 无梯度。
PPO mask 与空监督检查 2 情形 · 通过	0.000e+00	屏蔽位置梯度为 0；全空 mask 显式报错。
GAE 递推对显式展开 30 情形 · 通过	4.441e-16	不同轨迹长度及中间边界；逐项求和与倒序递推一致。
GAE 终止与外部截断 2 情形 · 通过	0.000e+00	相同 r/V 下，真实终止 advantage=-1，保留 bootstrap 的截断 advantage=2。
GRPO 组均值与零方差 2 情形 · 通过	0.000e+00	组内优势和近 0；同奖励组优势为 0，不声称独立 KL 项也无梯度。
DPO 零 margin 与方向 3 情形 · 通过	0.000e+00	margin=0 时 loss=ln2；提升 chosen logprob 降低 loss。
离散 KL 的 k1/k3 3 情形 · 通过	2.776e-17	共享支持集，精确 KL=0.192744757022；单样本 k1 可为负。

这些结果证明什么？

在所列输入、dtype 与边界条件下，代码输出和手工数学参考一致；相关梯度方向通过 autograd 对照。它们能发现符号、shift、归一化与边界错误，但不能证明完整系统在任意输入下正确。

没有验证的内容

没有进行 CUDA kernel 数值回归、多 GPU collective 执行、训练收敛比较、吞吐或成本测量。CPU 模拟 rank 的梯度平均，不等于实际启动了多进程 DDP。所有硬件选型与调参建议仍需在目标系统上运行。

> 复现时，先在小规模 float64 / FP32 reference 上检查语义，再进入 BF16/FP8 和分布式路径。误差阈值要结合数值尺度与下游质量，不要拿这页 float64 误差阈值直接约束低精度大模型。

附录 C / PRACTICE PATH

怎么用这本书准备面试

三条连续追问链

主线	建议题号	应当交付的能力
算法链	09 → 10 → 05 → 13 → 14 → 15	说清 PPO 一轮、clip 梯度、GAE 和 GRPO 的学习信号。
一致性链	20 → 21 → 22 → 23 → 24 → 25	区分四种策略，设计日志、correction 与权重发布。
项目链	29 → 30 → 34 → 35 → 39 → 40	从任务奖励到 held-out 评测，再到资源预算与证据。

一场 45 分钟的自测

前 5 分钟讲一个真实项目：问题、约束、基线、你的改动与验证。接下来 15 分钟选一条追问链，不看答案口述并推导；再用 15 分钟完成一道代码或预算题；最后 10 分钟让对方改变一个假设，看你是否能调整方案。

给答案打分，而不是给自己贴标签

层次	检查方法
1 · 定义	能否不用框架名字堆砌，准确说明输入、输出和目标。
2 · 推导	能否写出关键公式，说明归一化、边界和假设。
3 · 实现	能否定位 mask、张量布局、通信组或状态生命周期。
4 · 验证	能否设计对照、监控失败信号，解释哪些事实尚未验证。

项目表达模板

“原问题在什么条件下出现 → 我如何测量 → 排除了哪些原因 → 做了什么改动 → 正确性/质量如何验证 → 端到端收益与代价是什么”。没有实测的数据明确称作估算，不把本册教学配置包装成自己的项目经历。

> 复习的终点不是逐字背完：应当能在 batch 变大、序列变长、并行组变化、奖励失效或一个 rank 掉线时，重新推导系统行为。

来源档案 / INTERVIEW RECORDS

公开面经与题库

公司名为原作者或转载者的归属描述。A/B 不是面试难度评级；原帖里的答案、招聘推断和营销内容未自动采纳。

[I01] 蔚来｜大模型强化学习训练一面

A：作者自述；正文写面试日 2026-03-16。

公开主题：PPO/GAE、GRPO、verl/Ray、Agentic RL 与奖励稀疏。

关联题号：01、02、03、05、07、08、09、11、13、18、26、28、29、34、35、39、40（同类主题映射，非逐字题面）。

https://www.nowcoder.com/discuss/863132498270695424

[I02] 百度｜大模型后训练一面

A：作者自述；正文写面试日 2026-03-18。

公开主题：PPO clip、重要性采样、GAE、GRPO 归一化、分布式显存。

关联题号：02、03、04、05、10、12、13、14、15、19、20、24、31、36、37（同类主题映射，非逐字题面）。

https://www.nowcoder.com/discuss/863890669662674944

[I03] 字节｜大模型算法实习记录

A：作者自述；页面显示 03-19，年份不据此推定。

公开主题：在线/离线 RL、PPO、reference model、Attention。

关联题号：01、04、07、09、10（同类主题映射，非逐字题面）。

https://www.nowcoder.com/feed/main/detail/ad582f3774454b59869536b20299ecbb

[I04] B站｜算法二面记录

A：作者自述；页面发布日 2025-09-15，非确认面试日。

公开主题：DPO 动机、β、数据质量、PPO 稀疏奖励。

关联题号：04、08、16、17、38（同类主题映射，非逐字题面）。

https://www.nowcoder.com/feed/main/detail/f2a8e9c26b7b4915b96400287beb1255

[I07] 匿名多场｜AI Infra 复盘

A：多场自述；公司匿名，显示 03-03，不推定年份。

公开主题：LoRA、CUDA、TMA、FlashAttention、torchrun 与项目追问。

关联题号：40（同类主题映射，非逐字题面）。

https://www.nowcoder.com/feed/main/detail/d695614a06424c148c04586ac3a66e78

[I08] 字节｜大模型暑期面经主题整理

B：带推广的记录；显示 05-08，年份未确认。

公开主题：奖励、critic、KL、GRPO 不收敛、长对话与评估。

关联题号：09、11、12、18、29、30、34（同类主题映射，非逐字题面）。

https://www.nowcoder.com/feed/main/detail/0a36b958ef864c8db004bd45694ec9a6

来源档案 / INTERVIEW RECORDS

公开面经与题库 · 续

[I09] 快手｜大模型算法一面主题页

B：公开部分可读、部分付费；只用题目主题，不采信其答案。

公开主题：DPO 数据/β、GRPO group 与 old policy。

关联题号：13、14、16、17、38（同类主题映射，非逐字题面）。

https://www.nowcoder.com/discuss/876503715031572480

[I11] 字节大模型岗｜跨平台汇总

B：CSDN 汇编，宣称汇集多平台；不按每个平台重复计次。

公开主题：SFT、PPO/DPO/GRPO、数据、训练与推理显存；仅供主题发现。

关联题号：15、19、30、35、36（同类主题映射，非逐字题面）。

https://blog.csdn.net/qq_45717425/article/details/160315245

[I12] MiniMax｜AI Infra 题目转载

B：AIInfraGuide 转载；2026-04-17 为站点日期，不当作面试日。

公开主题：SFT loss 手写、LoRA、PPO/DPO/GRPO 与工具调用。

关联题号：08、16、29、38、40（同类主题映射，非逐字题面）。

https://caomaolufei.github.io/AIInfraGuide/interview/minimax-ai-infra-一面/

[I13] LLM RL Infra Interview｜开源题库

B：社区题库，非独立面试记录。

公开主题：训练/RL Infra 知识范围检查；不继承其“高频”统计。

关联题号：20、21、22、23、24、25、26、27、28、31、32、33、36、37、39（同类主题映射，非逐字题面）。

https://github.com/XFWang522/llm-rl-infra-interview

[I14] RL 面试 35 题｜社区整理

B：多级转述；页面标注 2026-06-06。

公开主题：RL 工程追问线索，不作为算法事实来源。

关联题号：12、20、21、22、23、25、27、32、33（同类主题映射，非逐字题面）。

https://iviews.info/posts/2026/06/rl-interview-questions-2026.html

原始依据 / PRIMARY REFERENCES

论文与官方技术依据

算法以论文与作者项目为依据；实现细节以链接所示官方版本为准。网页访问日 2026-10-31。代码附录实际运行版本另见验证页，不与最新在线文档版本混为一谈。

[R01] Spinning Up：RL 基本概念与 Bellman 方程
https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

[R02] Spinning Up：策略梯度推导
https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html

[R03] Proximal Policy Optimization Algorithms
https://arxiv.org/abs/1707.06347

[R04] Spinning Up：PPO-Clip
https://spinningup.openai.com/en/latest/algorithms/ppo.html

[R05] Generalized Advantage Estimation
https://arxiv.org/abs/1506.02438

[R06] Gymnasium：Termination / Truncation
https://gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/

[R07] Playing Atari with Deep Reinforcement Learning
https://arxiv.org/abs/1312.5602

[R08] Deep Reinforcement Learning with Double Q-learning
https://arxiv.org/abs/1509.06461

[R09] Spinning Up：Soft Actor-Critic
https://spinningup.openai.com/en/latest/algorithms/sac.html

[R10] InstructGPT：Training Language Models with Human Feedback
https://arxiv.org/abs/2203.02155

[R11] DeepSeekMath：GRPO 原始算法
https://arxiv.org/html/2402.03300v3

[R12] Direct Preference Optimization
https://arxiv.org/html/2305.18290v3

[R13] TRL：GRPO Trainer
https://huggingface.co/docs/trl/grpo_trainer

[R14] TRL：DPO Trainer
https://huggingface.co/docs/trl/dpo_trainer

原始依据 / PRIMARY REFERENCES

论文与官方技术依据 · 续

[R15] TRL：Reward Trainer
https://huggingface.co/docs/trl/reward_trainer

[R16] DeepSeek-R1 技术报告
https://arxiv.org/html/2501.12948

[R17] DAPO：算法与训练系统
https://arxiv.org/html/2503.14476v2

[R18] Understanding R1-Zero-Like Training / Dr. GRPO
https://arxiv.org/html/2503.20783v2

[R19] Group Sequence Policy Optimization
https://arxiv.org/html/2507.18071v2

[R20] verl：Rollout Correction
https://verl.readthedocs.io/en/latest/algo/rollout_corr.html

[R21] Megatron Core：MoE Router Replay 设计
https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/router_replay.html

[R22] HybridFlow：RLHF 分布式执行与重分片
https://arxiv.org/html/2409.19256v2

[R23] AReaL：异步 RL 系统
https://arxiv.org/html/2505.24298v5

[R24] verl：Agent Loop
https://verl.readthedocs.io/en/latest/advance/agent_loop.html

[R25] Evaluating Large Language Models Trained on Code
https://arxiv.org/abs/2107.03374

[R26] verl：GRPO 实现与 batch 参数
https://verl.readthedocs.io/en/latest/algo/grpo.html

[T05] PyTorch：Autograd mechanics
https://docs.pytorch.org/docs/2.15/notes/autograd.html

[T08] DeepSpeed：ZeRO 分阶段教程
https://www.deepspeed.ai/tutorials/zero/

原始依据 / PRIMARY REFERENCES

论文与官方技术依据 · 续

[T09] PyTorch：DDP 设计

https://docs.pytorch.org/docs/2.15/notes/ddp.html

[T10] PyTorch：FSDP2 教程

https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html

[T11] Megatron Core：Parallelism Strategies

https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/parallelism-guide.html

[T19] Transformer Engine：FP8 / MXFP8 primer

https://nvidia.github.io/TransformerEngine/examples/fp8_primer.html

[T22] PyTorch：Distributed Checkpoint

https://docs.pytorch.org/docs/2.15/distributed.checkpoint.html

[T23] PyTorch：Reproducibility

https://docs.pytorch.org/docs/2.15/notes/randomness.html

[T25] PyTorch：Profiler 教程

https://docs.pytorch.org/tutorials/recipes/recipes/profiler_recipe.html